Lab 6: Code and Branch Coverage Analysis

Student SRN: PES1UG24CS375
Module under test: amusement_park_ride_access.py (L6_L7_11 – Amusement Park Ride Access)
GitHub repository: https://github.com/Rishi-Jain-005/SE-Labs-PES1UG24CS375 (folder **LAB6**)

Summary

Measure	Starter test	Final result	Required	Test file
Line (code) coverage	48%	100%	≥ 90%	CodeCoverageTest_PES1UG24CS375_application.py
Branch coverage	45%	100% (40/40 branches)	≥ 90%	BranchCoverageTest_PES1UG24CS375_application.py

Commits (2 separate):
1. Add code (line) coverage tests - 100% line coverage
2. Add branch coverage tests - 100% branch coverage

Approach. The code coverage file exercises every statement: invalid input, closed groups, each ride type, unknown ride type, invalid and short heights, health warnings (roller coaster vs other rides), child pricing, fast lane, safety audit and zero-admission statuses, and all restraint-sensor outcomes. The branch coverage file makes every decision evaluate to both True and False and adds boundary cases (height 119/120, 4 vs 5 rejections, one vs two consecutive unlatched sensors, streak reset between sensor streams, default values for missing dictionary keys).

1. Code Coverage Result (Line Coverage)

Command: `coverage run -m pytest CodeCoverageTest_PES1UG24CS375_application.py` then `coverage report -m`

```
PS C:\Users\rishi\L6_L7_11_amusement_park_ride_access\Lab 6> coverage run -m pytest CodeCoverageTest_PES1UG24CS375_application.py
============================================================ test session starts =============================================================
platform win32 -- Python 3.12.5, pytest-8.3.5, pluggy-1.5.0
rootdir: C:\Users\rishi\L6_L7_11_amusement_park_ride_access\Lab 6
plugins: anyio-4.12.0
collected 16 items

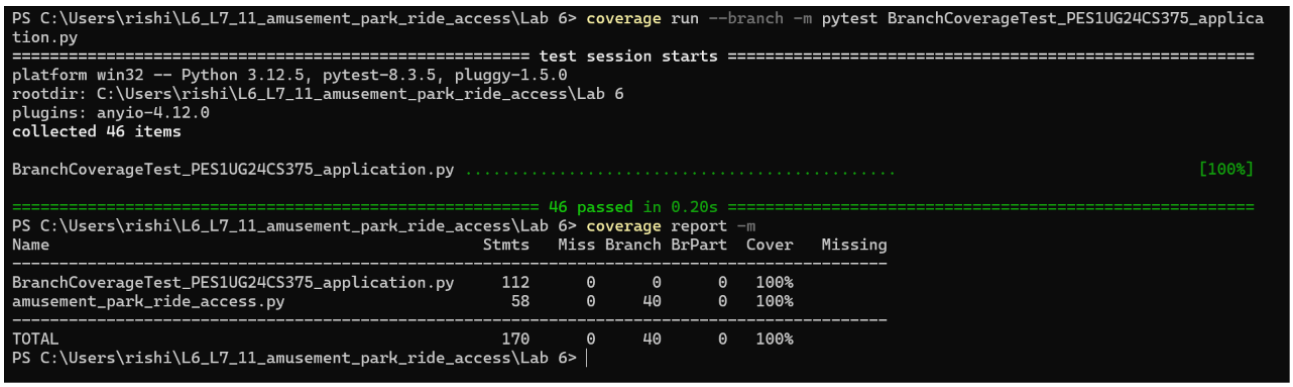
CodeCoverageTest_PES1UG24CS375_application.py ................                                                                      [100%]

============================================================= 16 passed in 0.16s =============================================================
PS C:\Users\rishi\L6_L7_11_amusement_park_ride_access\Lab 6> coverage report -m
Name                                                Stmts   Miss  Cover   Missing
---------------------------------------------------------------------------------
CodeCoverageTest_PES1UG24CS375_application.py          53      0   100%
amusement_park_ride_access.py                          58      0   100%
---------------------------------------------------------------------------------
TOTAL                                                 111      0   100%
```

Result: 16 tests passed; amusement_park_ride_access.py – 58 statements, 0 missed, **100%** line coverage.

2. Branch Coverage Result

Command: `coverage run --branch -m pytest BranchCoverageTest_PES1UG24CS375_application.py` then `coverage report -m`



Result: 46 tests passed; amusement_park_ride_access.py – 58 statements, 0 missed, 40 branches, 0 partial, **100%** branch coverage.

3. Test File: CodeCoverageTest_PES1UG24CS375_application.py

```
"""
Lab 6 - Code (Line) Coverage tests
SRN: PES1UG24CS375
Target module: amusement_park_ride_access.py
Goal: execute every line of the module at least once.
"""
import pytest
from amusement_park_ride_access import process_ride_admissions, inspect_safety_restraints


def make_group(ride_type="family", patrons=None, is_open=True, fast_lane=False):
    return {
        "is_open": is_open,
        "ride_type": ride_type,
        "has_fast_lane": fast_lane,
        "patrons": patrons if patrons is not None else [],
    }


def patron(height=150, health=False, child=False):
    return {"height_cm": height, "health_warning": health, "is_child": child}


# ---------- process_ride_admissions ----------

def test_single_adult_family_ride():
    result = process_ride_admissions([make_group("family", [patron()])], 5.0)
    assert result == {"total_revenue": 10.0, "status": "COMPLETED"}


def test_invalid_groups_type_raises():
    with pytest.raises(ValueError):
        process_ride_admissions("not a list", 5.0)


def test_closed_group_is_skipped():
    result = process_ride_admissions([make_group(is_open=False, patrons=[patron()])], 5.0)
    assert result == {"total_revenue": 0.0, "status": "ZERO_ADMISSIONS"}


def test_roller_coaster_adult():
    result = process_ride_admissions([make_group("roller_coaster", [patron()])], 5.0)
    assert result["total_revenue"] == 25.0


def test_water_ride_adult():
    result = process_ride_admissions([make_group("water_ride", [patron()])], 5.0)
    assert result["total_revenue"] == 18.0


def test_unknown_ride_type_rejected():
    result = process_ride_admissions([make_group("bumper_cars", [patron()])], 5.0)
    assert result["status"] == "ZERO_ADMISSIONS"


def test_invalid_height_rejected():
    result = process_ride_admissions([make_group("family", [patron(height=0)])], 5.0)
    assert result["status"] == "ZERO_ADMISSIONS"


def test_health_warning_on_roller_coaster_voids_group():
    patrons = [patron(), patron(health=True), patron()]
    result = process_ride_admissions([make_group("roller_coaster", patrons)], 5.0)
    assert result == {"total_revenue": 0.0, "status": "ZERO_ADMISSIONS"}


def test_health_warning_on_family_ride_only_rejects_that_patron():
    result = process_ride_admissions([make_group("family", [patron(), patron(health=True)])], 5.0)
    assert result == {"total_revenue": 10.0, "status": "COMPLETED"}


def test_too_short_patron_rejected():
    result = process_ride_admissions([make_group("family", [patron(height=119)])], 5.0)
    assert result["status"] == "ZERO_ADMISSIONS"


def test_child_gets_discount():
    result = process_ride_admissions([make_group("roller_coaster", [patron(child=True)])], 5.0)
    assert result["total_revenue"] == 15.0


def test_fast_lane_fee_added():
    result = process_ride_admissions([make_group("family", [patron()], fast_lane=True)], 5.0)
    assert result["total_revenue"] == 15.0


def test_safety_audit_when_more_than_four_rejections():
```

```
    patrons = [patron(height=100) for _ in range(5)] + [patron()]
    result = process_ride_admissions([make_group("family", patrons)], 5.0)
    assert result == {"total_revenue": 10.0, "status": "SAFETY_AUDIT"}


# ---------- inspect_safety_restraints ----------

def test_restraints_no_data():
    assert inspect_safety_restraints([]) == "NO_DATA"


def test_restraints_emergency_lock():
    assert inspect_safety_restraints([[1, 0, 0]]) == "EMERGENCY_LOCK"


def test_restraints_safe_with_streak_reset():
    assert inspect_safety_restraints([[0, 1, 0, 1]]) == "SAFE"
```

4. Test File: BranchCoverageTest_PES1UG24CS375_application.py

```
"""
Lab 6 - Branch Coverage tests
SRN: PES1UG24CS375
Target module: amusement_park_ride_access.py
Goal: make every decision evaluate to BOTH True and False, including boundaries.
"""
import pytest
from amusement_park_ride_access import process_ride_admissions, inspect_safety_restraints


def make_group(ride_type="family", patrons=None, is_open=True, fast_lane=False):
    return {
        "is_open": is_open,
        "ride_type": ride_type,
        "has_fast_lane": fast_lane,
        "patrons": patrons if patrons is not None else [],
    }


def patron(height=150, health=False, child=False):
    return {"height_cm": height, "health_warning": health, "is_child": child}


# ---------- Input validation: `not isinstance(...) or fee < 0` ----------

def test_groups_not_list_raises():
    with pytest.raises(ValueError):
        process_ride_admissions(None, 5.0)

def test_negative_fee_raises():
    with pytest.raises(ValueError):
        process_ride_admissions([], -0.01)

def test_zero_fee_and_empty_list_is_valid():
    assert process_ride_admissions([], 0) == {"total_revenue": 0.0, "status": "ZERO_ADMISSIONS"}


# ---------- is_open True / False ----------

def test_open_group_is_processed():
    assert process_ride_admissions([make_group(patrons=[patron()])], 5)["total_revenue"] == 10.0

def test_closed_group_skipped_but_open_group_still_counted():
    groups = [make_group(is_open=False, patrons=[patron()]), make_group(patrons=[patron()])]
    assert process_ride_admissions(groups, 5)["total_revenue"] == 10.0

def test_missing_is_open_defaults_to_closed():
    assert process_ride_admissions([{"ride_type": "family", "patrons": [patron()]}], 5)["status"] == "ZERO_ADMISSIONS"


# ---------- ride_type if/elif/elif/else ----------

@pytest.mark.parametrize("ride_type, expected", [
    ("roller_coaster", 25.0),
    ("water_ride", 18.0),
    ("family", 10.0),
])
def test_each_valid_ride_type_price(ride_type, expected):
    assert process_ride_admissions([make_group(ride_type, [patron()])], 5)["total_revenue"] == expected

def test_missing_ride_type_defaults_to_family():
    group = {"is_open": True, "patrons": [patron()]}
    assert process_ride_admissions([group], 5)["total_revenue"] == 10.0

def test_unknown_ride_type_counts_one_rejection_and_skips_group():
    result = process_ride_admissions([make_group("ferris_wheel", [patron(), patron()], fast_lane=True)], 5)
    assert result == {"total_revenue": 0.0, "status": "ZERO_ADMISSIONS"}


# ---------- patron loop: empty / non-empty ----------

def test_group_with_no_patrons():
    assert process_ride_admissions([make_group(patrons=[])], 5)["status"] == "ZERO_ADMISSIONS"

def test_missing_patrons_key_defaults_to_empty():
    assert process_ride_admissions([{"is_open": True, "ride_type": "family"}], 5)["status"] == "ZERO_ADMISSIONS"

def test_empty_group_with_fast_lane_still_charges_fee():
    result = process_ride_admissions([make_group(patrons=[], fast_lane=True)], 7.5)
    assert result == {"total_revenue": 7.5, "status": "COMPLETED"}


# ---------- height <= 0 ----------

@pytest.mark.parametrize("h", [0, -5])
def test_non_positive_height_rejected(h):
    assert process_ride_admissions([make_group(patrons=[patron(height=h)])], 5)["status"] == "ZERO_ADMISSIONS"
```

```
def test_height_one_cm_passes_first_check_but_fails_min_height():
    assert process_ride_admissions([make_group(patrons=[patron(height=1)])], 5)["status"] == "ZERO_ADMISSIONS"

def test_missing_height_defaults_to_zero_and_is_rejected():
    result = process_ride_admissions([make_group(patrons=[{"is_child": False}])], 5)
    assert result["status"] == "ZERO_ADMISSIONS"


# ---------- health warning: True/False x roller_coaster / other ----------

def test_health_warning_roller_coaster_breaks_and_zeroes_group_total():
    patrons = [patron(), patron(), patron(health=True), patron()]
    result = process_ride_admissions([make_group("roller_coaster", patrons)], 5)
    assert result["total_revenue"] == 0.0

def test_health_warning_roller_coaster_still_charges_fast_lane():
    result = process_ride_admissions([make_group("roller_coaster", [patron(), patron(health=True)], fast_lane=True)], 5)
    assert result == {"total_revenue": 5.0, "status": "COMPLETED"}

@pytest.mark.parametrize("ride_type, price", [("water_ride", 18.0), ("family", 10.0)])
def test_health_warning_non_coaster_continues_with_next_patron(ride_type, price):
    result = process_ride_admissions([make_group(ride_type, [patron(health=True), patron()])], 5)
    assert result == {"total_revenue": price, "status": "COMPLETED"}

def test_health_warning_takes_priority_over_short_height():
    # short AND health warning -> still counts as one rejection, handled by the health branch
    result = process_ride_admissions([make_group("family", [patron(height=100, health=True), patron()])], 5)
    assert result["total_revenue"] == 10.0


# ---------- height < 120 boundary ----------

def test_height_119_rejected():
    assert process_ride_admissions([make_group(patrons=[patron(height=119)])], 5)["status"] == "ZERO_ADMISSIONS"

def test_height_120_accepted():
    assert process_ride_admissions([make_group(patrons=[patron(height=120)])], 5)["total_revenue"] == 10.0


# ---------- is_child True / False ----------

def test_child_pays_60_percent():
    assert process_ride_admissions([make_group("water_ride", [patron(child=True)])], 5)["total_revenue"] == pytest.approx(10.8)

def test_adult_pays_full_price():
    assert process_ride_admissions([make_group("water_ride", [patron(child=False)])], 5)["total_revenue"] == 18.0

def test_mixed_family_child_and_adult():
    result = process_ride_admissions([make_group("roller_coaster", [patron(), patron(child=True)])], 5)
    assert result["total_revenue"] == 40.0


# ---------- has_fast_lane True / False ----------

def test_fast_lane_true():
    assert process_ride_admissions([make_group(patrons=[patron()], fast_lane=True)], 5)["total_revenue"] == 15.0

def test_fast_lane_false():
    assert process_ride_admissions([make_group(patrons=[patron()], fast_lane=False)], 5)["total_revenue"] == 10.0

def test_missing_fast_lane_defaults_false():
    group = {"is_open": True, "ride_type": "family", "patrons": [patron()]}
    assert process_ride_admissions([group], 5)["total_revenue"] == 10.0


# ---------- final status: rejected > 4 / revenue == 0 / completed ----------

def test_exactly_four_rejections_is_not_audit():
    patrons = [patron(height=100)] * 4 + [patron()]
    assert process_ride_admissions([make_group(patrons=patrons)], 5)["status"] == "COMPLETED"

def test_five_rejections_triggers_audit():
    patrons = [patron(height=100)] * 5 + [patron()]
    assert process_ride_admissions([make_group(patrons=patrons)], 5)["status"] == "SAFETY_AUDIT"

def test_audit_takes_priority_over_zero_admissions():
    patrons = [patron(height=100)] * 5
    result = process_ride_admissions([make_group(patrons=patrons)], 5)
    assert result == {"total_revenue": 0.0, "status": "SAFETY_AUDIT"}

def test_rejections_accumulate_across_groups():
    groups = [make_group(patrons=[patron(height=100)] * 3), make_group("bad_type"), make_group(patrons=[patron(height=0)])]
    assert process_ride_admissions(groups, 5)["status"] == "SAFETY_AUDIT"

def test_revenue_is_rounded_to_two_decimals():
    result = process_ride_admissions([make_group("water_ride", [patron(child=True)] * 3)], 0.333)
    assert result["total_revenue"] == 32.4

def test_zero_admissions_status():
    assert process_ride_admissions([make_group(patrons=[patron(height=100)])], 5)["status"] == "ZERO_ADMISSIONS"


# ---------- inspect_safety_restraints ----------

@pytest.mark.parametrize("data", [[], None])
```

```
def test_restraints_no_data(data):
    assert inspect_safety_restraints(data) == "NO_DATA"

def test_restraints_two_consecutive_zeros_triggers_lock():
    assert inspect_safety_restraints([[0, 0]]) == "EMERGENCY_LOCK"

def test_restraints_single_zero_is_safe():
    assert inspect_safety_restraints([[0]]) == "SAFE"

def test_restraints_non_consecutive_zeros_reset_streak():
    assert inspect_safety_restraints([[0, 1, 0, 1, 0]]) == "SAFE"

def test_restraints_all_latched_is_safe():
    assert inspect_safety_restraints([[1, 1, 1]]) == "SAFE"

def test_restraints_empty_sensor_list_is_safe():
    assert inspect_safety_restraints([[]]) == "SAFE"

def test_restraints_streak_resets_between_streams():
    # trailing 0 of stream 1 must not combine with leading 0 of stream 2
    assert inspect_safety_restraints([[1, 0], [0, 1]]) == "SAFE"

def test_restraints_lock_found_in_later_stream():
    assert inspect_safety_restraints([[1, 1], [1, 0, 0]]) == "EMERGENCY_LOCK"
```